

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Constraint-Based Problem Solving

COURSE NAME: Cloud Computing and
Big Data Analytics

COURSE CODE: CSA1504

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.*
(BL5)

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 50

Code of Conduct

I S Jefna Princy- 192572058 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

Assessment Tasks

Constraint-Based Problem Solving

1. Design an HDFS-based storage solution for a media platform under the constraints that data availability must remain above 99.9%, replication factor cannot exceed 3, and storage growth must stay cost efficient. Justify your design.
2. A MapReduce workflow must process log data within a 20-minute batch window using limited cluster nodes. Propose a job design under these constraints and explain task distribution.
3. A YARN-managed cluster supports ETL, reporting, and machine learning jobs. Design a scheduler policy under the constraints of fairness, priority for ETL, and recovery from node failure.
4. An enterprise needs to ingest data from SAN, NAS, and operational databases into a big data platform while maintaining backup reliability and minimal duplication. Propose a constrained architecture and justify each component.
5. Design an end-to-end Hadoop ecosystem solution under the constraints of secure data movement, high availability, and integration with Apache NiFi or ETL pipelines. Explain how your design satisfies all constraints.

Constraint-Based Problem Solving Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Constraints	25%	All technical constraints are identified and prioritized accurately	Most constraints identified	Some constraints identified	Constraints poorly understood

System Design Quality	25%	Design is robust, feasible, and aligned to constraints	Reasonable design	Basic design with gaps	Weak design
Application of Hadoop Concepts	20%	Uses HDFS, MapReduce, YARN, and integration concepts effectively	Mostly effective use	Partial use of concepts	Weak concept application
Justification	15%	Strong technical reasoning for all choices	Good justification	Basic justification	Little justification
Clarity	15%	Clear, well-structured, and professional answer	Mostly clear	Somewhat unclear	Poorly structured

CONSTRAINT-BASED PROBLEM SOLVING

1. HDFS-Based Storage Solution for a Media Platform

A media platform stores a very large amount of data such as videos, images, audio files, user information, and application logs. Since the platform needs high availability and cost-efficient storage, **Hadoop Distributed File System (HDFS)** can be used as the main storage layer.

Constraints

The major constraints are **data availability above 99.9%, replication factor not exceeding 3, and cost-efficient storage growth**. The system should also be able to handle increasing amounts of media data without affecting performance. A replication factor of **3** can be selected because it provides fault tolerance while staying within the specified limit. If one storage node fails, another replica can continue serving the data.

Proposed HDFS Design

The HDFS architecture consists mainly of a **NameNode** and multiple **DataNodes**. The NameNode maintains metadata about files and their locations, while DataNodes store the actual media blocks.

Large media files are divided into blocks and distributed across multiple DataNodes. Each block can have up to three replicas. The replicas should preferably be placed on different nodes or racks so that a hardware or rack failure does not make the data unavailable.

The NameNode should be protected using high-availability mechanisms so that failure of the primary metadata service does not make the entire storage system unavailable.

Cost-Efficient Storage

Since media data continuously increases, storage should be expanded by adding DataNodes instead of replacing the entire storage system. Frequently accessed media can be kept on high-performance storage, while older or less frequently accessed content can be moved to lower-cost storage.

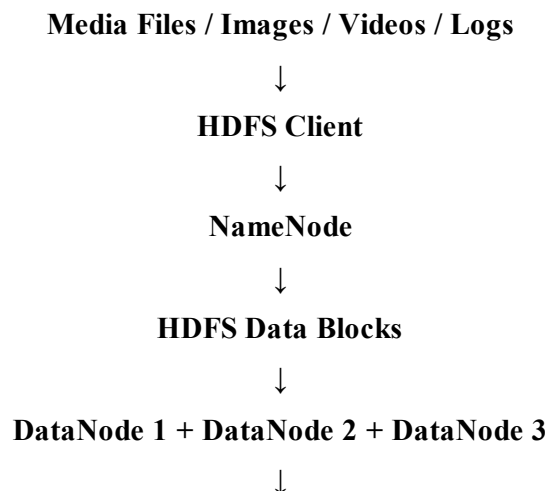
Compression can also reduce storage requirements for suitable data such as logs and metadata.

Availability and Fault Tolerance

HDFS automatically maintains replicas of data blocks. If one DataNode fails, another replica can be accessed. The system can also detect failed nodes and recreate missing replicas on healthy nodes.

Monitoring tools should continuously check node health, disk usage, and replication status.

Overall Flow



Replication + Fault Tolerance



Media Processing / Analytics

Justification

HDFS is suitable because it is designed for **large-scale distributed storage**. A maximum replication factor of three provides a good balance between reliability and storage cost. Horizontal expansion allows the platform to handle future storage growth.

Therefore, the proposed design satisfies the availability, replication, and cost constraints while providing scalable storage for the media platform.

2. MapReduce Workflow for Log Processing

A company needs to process large volumes of log data within a **20-minute batch window** while using a limited number of cluster nodes. The main challenge is to complete the processing within the given time without unnecessarily consuming cluster resources.

Constraints

The major constraints are:

- Processing must finish within **20 minutes**.
- The number of available cluster nodes is limited.
- Large volumes of log data must be processed.
- The system should distribute the workload efficiently.
- Failures should not cause the entire job to restart unnecessarily.

Proposed MapReduce Design

The log files can first be stored in **HDFS**. Each large log file is divided into blocks, which allows Hadoop to process different parts of the data in parallel.

The MapReduce job consists of **Mapper, Shuffle and Sort, and Reducer** stages.

The Mapper reads individual log records and extracts useful information such as timestamp, IP address, status code, URL, or error type.

For example, a Mapper could generate:

Error 500 → 1

Error 404 → 1

Error 500 → 1

During the Shuffle and Sort phase, Hadoop groups records with the same key.

The Reducer then calculates the total number of occurrences.

For example:

Error 500 → 2

Error 404 → 1

Job Distribution

Since the number of cluster nodes is limited, the input data should be divided into appropriate partitions. Hadoop can execute multiple Mapper tasks in parallel across the available nodes.

The number of reducers should be selected according to the available CPU and memory. Too many reducers may create

unnecessary overhead, while too few reducers may increase processing time.

Data Locality

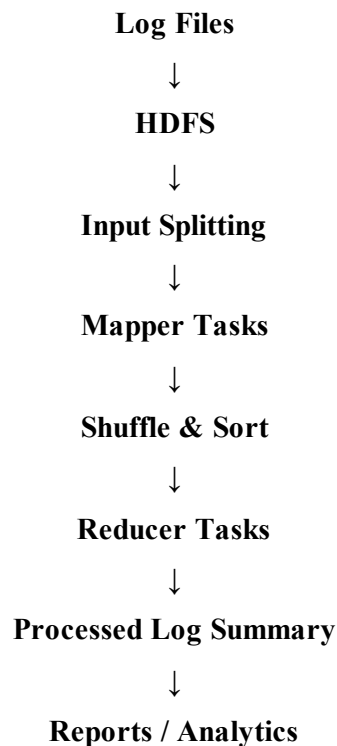
Hadoop attempts to run computation close to where the input data is stored. This reduces network communication and improves performance.

This is particularly useful when processing large log files because moving the entire dataset across the network would consume significant bandwidth.

Fault Handling

If a Map or Reduce task fails, Hadoop can automatically rerun the failed task on another available node. This provides reliability without restarting the entire job.

Overall Flow



Justification

MapReduce is appropriate because it divides a large processing task into smaller parallel tasks. Data locality reduces network overhead, while automatic task retry improves reliability.

The cluster can be tuned by adjusting the number of Mapper and Reducer tasks according to the available nodes. Therefore, the proposed design can meet the **20-minute processing constraint** while making efficient use of limited cluster resources.

3. YARN-Based Scheduler for ETL, Reporting and Machine Learning

A Hadoop cluster needs to support different workloads such as **ETL processing, reporting, and machine learning**. The main challenge is to allocate limited cluster resources fairly while giving appropriate priority to important jobs.

Constraints

The major constraints are:

- ETL jobs should receive higher priority when required.

- Resources should be shared fairly between different users and applications.
- Failed nodes should not permanently stop processing.
- Machine learning and reporting jobs should continue to receive resources.
- Cluster resources such as CPU and memory are limited.

Proposed YARN Architecture

YARN (Yet Another Resource Negotiator) can manage the cluster resources. The **ResourceManager** acts as the central resource manager, while **NodeManagers** manage resources on individual worker nodes.

Each submitted application receives an **ApplicationMaster**, which coordinates the execution of that application.

The cluster can be divided into logical queues such as:

- **ETL Queue**
- **Reporting Queue**
- **Machine Learning Queue**

ETL can be assigned higher priority because it may prepare data required by other applications. Reporting can receive medium priority, while long-running machine learning jobs can use available resources without completely blocking other workloads.

Fairness

A fair scheduling mechanism ensures that one workload does not continuously consume all available resources.

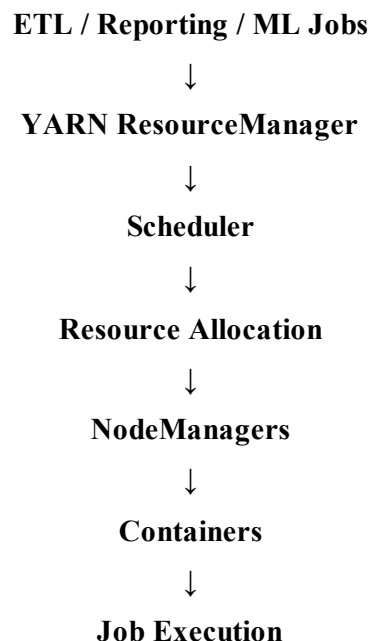
For example, if an ETL job requires a large amount of CPU, YARN can allocate resources to it according to its priority while still reserving resources for reporting and machine learning.

Recovery from Node Failure

If a worker node fails, YARN can detect the failure through the NodeManager heartbeat mechanism. Tasks running on the failed node can be restarted on another available node.

This prevents a single node failure from stopping the complete data processing system.

Overall Flow



Justification

YARN is suitable because it separates **resource management from application processing**. The scheduler can control

how CPU and memory are distributed between different workloads.

Priority-based scheduling can ensure important ETL operations receive appropriate resources, while fair resource allocation prevents reporting and machine learning jobs from being completely starved.

Thus, the design satisfies the requirements of **fairness, priority, and recovery from node failure**.

4. Constrained Architecture for SAN, NAS and Operational Databases

An enterprise needs to ingest data from different sources such as **SAN, NAS, and operational databases** into a big data platform. The system must maintain reliable backups while avoiding unnecessary duplication of data.

Constraints

The major constraints are:

- Data must be collected from multiple storage systems.
- Backup reliability must be maintained.
- Unnecessary data duplication should be minimized.
- Different data formats and sources must be supported.
- The architecture should be scalable.
- Data should remain available even during failures.

Proposed Architecture

The first layer consists of data sources such as **SAN storage, NAS storage, relational databases, and operational applications**.

Data ingestion tools can collect information from these different sources. Database records can be imported using connectors, while files from SAN and NAS can be transferred through suitable ingestion mechanisms.

The collected data can be stored in HDFS. Instead of maintaining multiple unnecessary copies at every stage, the system should use a controlled storage strategy.

HDFS Storage

HDFS can provide distributed storage and fault tolerance. A suitable replication factor can be selected according to the importance of the data.

For critical datasets, replication can provide protection against node failures. Less critical temporary data can have shorter retention periods.

Backup Strategy

Important data should be backed up using scheduled snapshots or a separate backup system. Backup policies should distinguish between critical and temporary data.

Data lifecycle management can automatically archive older data and remove expired temporary datasets.

Minimizing Duplication

Data should not be copied repeatedly between multiple processing stages. A centralized data lake can act as the main storage layer.

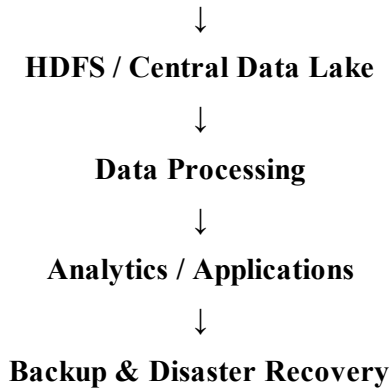
Metadata and cataloguing can identify existing datasets so that duplicate copies are not unnecessarily created.

Overall Flow

SAN / NAS / Operational Databases



Data Ingestion Layer



Justification

The proposed architecture provides a common storage and processing environment for heterogeneous data sources. HDFS provides distributed storage and fault tolerance, while lifecycle management controls storage growth. A centralized architecture reduces unnecessary duplication, while controlled replication and backups improve reliability. Therefore, the architecture balances **data availability, backup reliability, scalability, and storage efficiency**.

5. End-to-End Hadoop Ecosystem with Apache NiFi / ETL

An organization requires an end-to-end Hadoop ecosystem that supports **secure data movement, high availability, and integration with Apache NiFi or ETL pipelines**. The solution must efficiently move data from different sources into Hadoop for storage and analysis.

Constraints

The main constraints are:

- Data movement must be secure.
- The platform should remain highly available.
- Apache NiFi or ETL pipelines must be integrated.
- Large amounts of data must be processed.
- The system should support distributed storage and processing.
- Failure of individual components should not cause major data loss.

Data Ingestion

Apache NiFi can be used as the data ingestion and flow-management layer. It can collect data from databases, applications, files, APIs, and other sources.

NiFi processors can perform operations such as filtering, routing, transformation, and validation before the data enters Hadoop.

For example:

Database → NiFi → Validation → Transformation → HDFS

Hadoop Storage

After ingestion, data can be stored in HDFS. HDFS divides large files into blocks and distributes them across DataNodes. Replication provides fault tolerance, while the NameNode manages metadata and file locations.

Processing

MapReduce or **Apache Spark** can process the stored data. YARN manages the cluster resources and schedules applications.

Batch jobs can be used for large historical datasets, while suitable streaming technologies can handle continuously

arriving data.

Security

Secure data movement can be implemented using encrypted communication and authentication. Access permissions should restrict users from accessing unauthorized datasets.

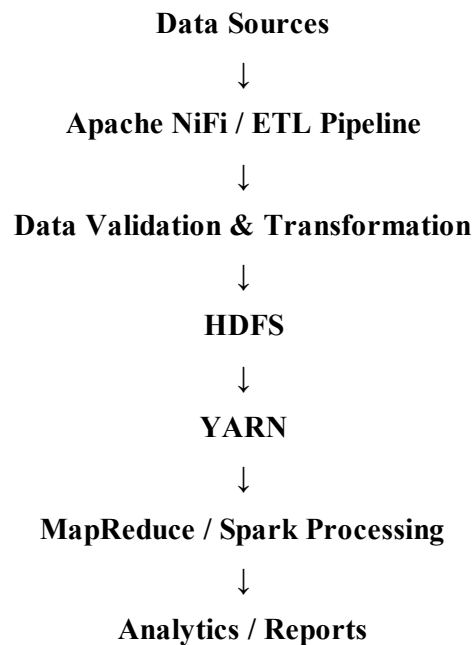
Sensitive data should be protected both during transmission and while stored in the Hadoop environment.

High Availability

Important Hadoop services should be configured with high-availability mechanisms. Data replication ensures that data remains available when a DataNode fails.

NiFi can also provide flow monitoring and recovery mechanisms to prevent data loss during pipeline failures.

Overall Flow



Justification

The proposed architecture combines the major Hadoop ecosystem components. **NiFi/ETL** handles data ingestion, **HDFS** provides distributed storage, **YARN** manages resources, and **MapReduce/Spark** performs processing.

Security mechanisms protect data movement, while HDFS replication and high-availability services improve system reliability.

Therefore, the design satisfies all three major constraints: **secure data movement, high availability, and integration with NiFi/ETL pipelines.**